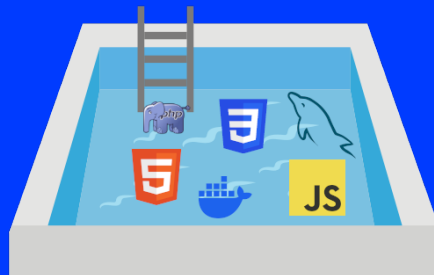




BOOTCAMP WEB

< DAY 03 - DIVING INTO HTML & CSS />



BOOTCAMP WEB

Responsive Design

Today, we will address responsive design, with a mobile-first approach.

Now that you know how to build an HTML page, let's see how to make it pretty and fit screens of all sizes: desktops, tablets, smartphones, etc...

It is an approach used to develop sites that offer an optimal reading and browsing experience to the user, regardless of their device (smartphone, tablet, reader, desktop, printer, television, ...)

Traditionally, a web project that adopts a responsive design strategy will tend to think of the desktop version of the site as a starting point. It will then be necessary to *reduce* this version as the size of the consultation materials decreases, according to a predefined grid. We call this **graceful degradation**.

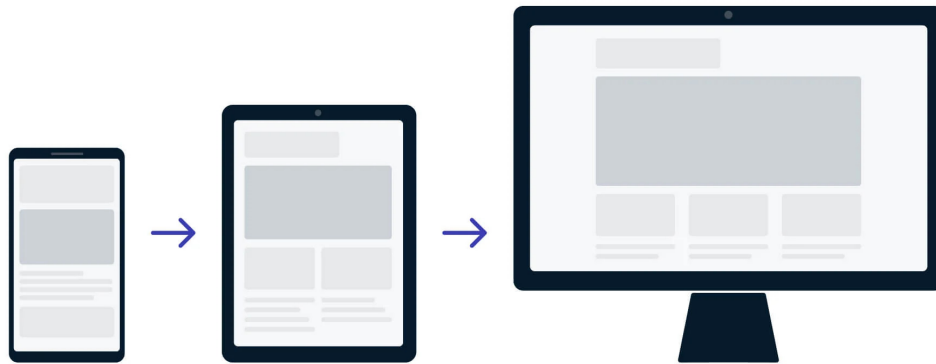
This seems logical, since it feels like the majority of the web traffic comes from desktop widescreens when it actually comes from mobile devices (and most particularly smartphones).

However, if you take the contents of a water bottle and pour it all into a glass of water...



Mobile first

Another way of doing things has been talked about: the **mobile first** approach.



As the name suggests, this approach consists of thinking about navigation, ergonomics and site content for the smallest consultation materials first, and then upscaling them. This is called **progressive enhancement**.

If this approach begins to have some success over graceful degradation, it is because it has some undeniable advantages that do not only concern mobile versions of websites.



You must use **flexbox** and the **viewport meta tag** for these exercises.



You'll need to find the browser feature (or use an extension) that allows you to change the viewport. Learn to use your browser's dev tools, they will be your best friends!



If you need a little break in your day, we suggest you to play:

- ✓ [CSS Diner](#);
- ✓ [CSS Grid garden](#);
- ✓ [CSS Grid attack](#);
- ✓ [Flexbox Froggy](#);
- ✓ [Flexbox Defense](#).
- ✓ [Flexbox Adventure](#)

Task01

Delivery: `./task01/index.html` and `./task01/style.css`

Create an **index.html** page and link it to a **style.css** file.
Structure your HTML with:

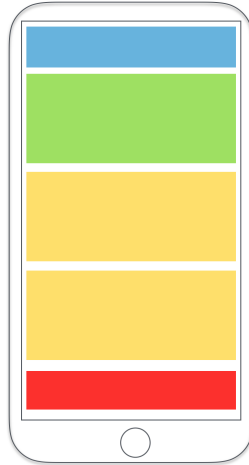
- ✓ a header containing a nav;
- ✓ a first section;
- ✓ a second section with six articles;
- ✓ a footer.

Task02 - Smartphone

Delivery: `./task02/index.html` and `./task02/style.css`

Reuse the structure and content from the previous task. You will now adapt the layout to fit for **smartphones** (devices with a maximum width of 767 px). It will be the default layout (mobile-first approach).

Your goal is to reproduce the layout shown in the reference image:



- ✓ The header (blue) appears at the top.
- ✓ The first section (green) comes next.
- ✓ Only the two first articles of the second section (yellow) are visible.
- ✓ The footer (red) appears at the bottom.
- ✓ Each colored area must be **stacked vertically**, taking the **full width** of the screen.

To achieve this:

- ✓ Use **Flexbox** to structure the layout.
- ✓ Apply the mobile styles only for screens up to **767 px wide**.
- ✓ Hide the remaining articles (starting from the third one).
- ✓ Make sure the visual result is close to the reference: there should be some spacing between the colored blocks and no unexpected white margins around the page.
- ✓ Use these color classes : `{.blue:#67B3DD, .green:#9EE062, .yellow:#FEDF6B, .red: #FC302D}`



To check your work more easily, add short text inside each area (e.g. "navigation", "first section", "article 1", "footer", etc.) so that you can clearly see which block is which when testing your page.

Task03 — Tablet Layout

Delivery: `./task03/index.html` and `./task03/style.css`

Reuse the structure and content from the previous task.

You will now adapt the layout for **tablets** (devices with a resolution width between **768 px and 1023 px**).

Your goal is to reproduce the layout shown in the reference image:



- ✓ The **header** (blue) appears at the top, aligned to the left.
- ✓ The **first section** (green) comes next, taking the full width.
- ✓ In the second section, **four articles** are now visible:
 - The first two (yellow) appear side by side,
 - followed by the next two (purple) also side by side.
- ✓ The **footer** (red) remains at the bottom and takes the full width.
- ✓ Each colored area must be clearly visible and separated by some spacing.

To achieve this:

- ✓ Keep your **mobile-first** base CSS.
- ✓ Add a **media query** that targets screens between `768px` and `1023px`.
- ✓ You must use **Flexbox** to control the layout and spacing of the sections and articles.
- ✓ Only the first **four** articles should be visible in this version.
- ✓ The color classes remain the same as before (with the addition of `.purple: #6B2683`):



Compared to the smartphone version, **two new articles appear** (the purple ones). Only the last two articles remain hidden in this version.



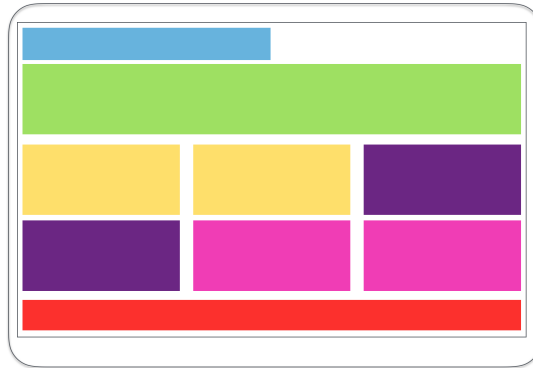
Use the same spacing logic as before (margins, padding or gap). Focus on the visual result — it should match the reference layout for tablet width.

Task04 - Desktop

Delivery: `./task04/index.html` and `./task04/style.css`

Reuse the structure and content from the previous task.

You will now adapt your layout for **desktop screens**, i.e. devices with a **minimum width of 1024px**.



To achieve this:

- ✓ All six articles must now be **visible**.
- ✓ You must continue to use **Flexbox** to align and distribute the content horizontally.
Articles must automatically wrap into **three columns** using **Flexbox properties**, without using Grid or fixed positioning.
- ✓ The `header` still takes half of the screen width.
- ✓ The background colors remain the same as before.
- ✓ The color classes remain the same as before (with the addition of `.pink: #F03DB5`):

You should obtain a layout similar to the provided desktop mockup.



This is the continuation of your responsive design work.

Make sure your layout automatically adapts when resizing the browser window:

- ✓ `< 768px`: vertical layout, 2 visible articles;
- ✓ `768px-1023px`: 2-column layout, 4 visible articles;
- ✓ `≥ 1024px` : 3-column layout, all 6 articles visible.

Task05

Delivery: `./task05/index.html`



For the next exercises, only use the **MaterializeCSS** framework.
Do not create any custom CSS file or rule.
The HTML `style` attribute is forbidden.

These titles must be correctly implemented: LASTNAME FIRSTNAME, Heading, About me, Story, Profile, Skills, Portfolio, Last projects, Contact.



You must include **MaterializeCSS** from a CDN (use version 2 or above).
You can find the documentation here: <https://materializeweb.com/>

Step A

Create an HTML page `index.html` and include the **MaterializeCSS** CDN inside.
Apply a light grey background color to the page using Materialize classes.
Add a `div` as wide as the page, with a background color of your choice.
Inside this `div`, create **two titles**:

- ✓ one containing your **lastname and firstname** in white;
- ✓ one containing your **profile heading** in grey with a white background.

Step B

Create a `div` container containing a row with title "About me". Inside this row, put a `div` which is:

- ✓ 12 columns wide for a small-sized screen;
- ✓ 4 columns wide for a medium to large screen.

Add to it a "Story" title, and a quick explanation about your professional background.



Use the [Materialize grid system](#) for this layout.

Step C

Inside the **About me** row, add another `div` of the same size as the "Story" one, titled **Profile**. Its background color is the same as the one you chose earlier. Inside it, create a **Materialize card** with:

- ✓ white inner text;
- ✓ a **responsive round profile picture** using the classes;
- ✓ a card's actions containing links to your **GitHub** and **LinkedIn** profiles.



Materialize has a lot of useful classes to help you with styling (the profil picture for instance). Use them!

Step D

Add a third `div` of the same size, titled **Skills**, next to the previous ones. Inside it, insert a **table** showing the languages and frameworks you know. Each row should include a short comment about each skill. Every two rows, one should have a **grey background** (use Materialize color classes).

Step E

Create two new rows, titled **Portfolio** and **Last projects**. Each row must contain **three images**, using Materialize responsive image classes.

Step F

Create a row titled **Contact**. Inside it, add a new row containing a **full-width form** including:

- ✓ a first row with two inputs ("Lastname" and "Firstname"), each taking **half** the row;
- ✓ a second row containing a full-width **email** input;
- ✓ a last row containing a **textarea** with a "Your message" placeholder.



All titles and buttons must be **centered** using a Materialize class.

v 4.2.2

{EPITECH}